

concordance=TRUE

CCMnetpy Demonstration Vignette

Ravi Goyal

December 3, 2025

1 Example 1: Edge Count with Negative Binomial Prior

```
reticulate::use_condaenv("ccmnet_env", required = TRUE)

library(reticulate)
library(CCMnetpy)
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse
2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2    3.5.2      v tibble     3.3.0
## v lubridate  1.9.4      v tidyr      1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts()
--
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to
force all conflicts to become errors

library(igraph)

##
## Attaching package: 'igraph'
##
## The following objects are masked from 'package:lubridate':
##
##   %--%, union
##
## The following objects are masked from 'package:dplyr':
##
```

```

##      as_data_frame, groups, union
##
## The following objects are masked from 'package:purrr':
##
##      compose, simplify
##
## The following object is masked from 'package:tidyr':
##
##      crossing
##
## The following object is masked from 'package:tibble':
##
##      as_data_frame
##
## The following objects are masked from 'package:stats':
##
##      decompose, spectrum
##
## The following object is masked from 'package:base':
##
##      union

# Initialize CCMnet Python code
CCMnet_python_setup()

population <- 100
covPattern <- rep(0L, population)

# Target statistic: edge count
Network_stats <- list("Edge")
Prob_Distr     <- list("NP")

# Negative binomial prior on number of edges
Prob_Distr_Params <- vector("list", 2)
Prob_Distr_Params[[1]] <- dnbinom(
  0:choose(population, 2),
  size = 1.017340,
  mu    = 6.192894
)

# Run a small MCMC chain
result <- CCMnetpy::CCMnet_constr(
  Network_stats = Network_stats,
  Prob_Distr    = Prob_Distr,
  Prob_Distr_Params = Prob_Distr_Params,
  G=NULL,

```

```

P=NULL,
samplesize      = 1000L,
burnin           = 50000L,
interval         = 10000L,
statsonly        = TRUE,
population       = population,
covPattern       = covPattern,
bayesian_inference = FALSE,
Ia = NULL,
Il = NULL,
R = NULL,
epi_params = NULL,
print_calculations = FALSE,
use_G = FALSE,
outfile = "none",
partial_network = as.integer(0),
obs_nodes = NULL,
MH_proposal_type = "TNT"
)

g      <- result[[1]]
stats <- result[[2]]

# Prepare data for density plot
df_sampled <- data.frame(
  value = stats[,population + 1],
  type  = "MCMC Sample"
)

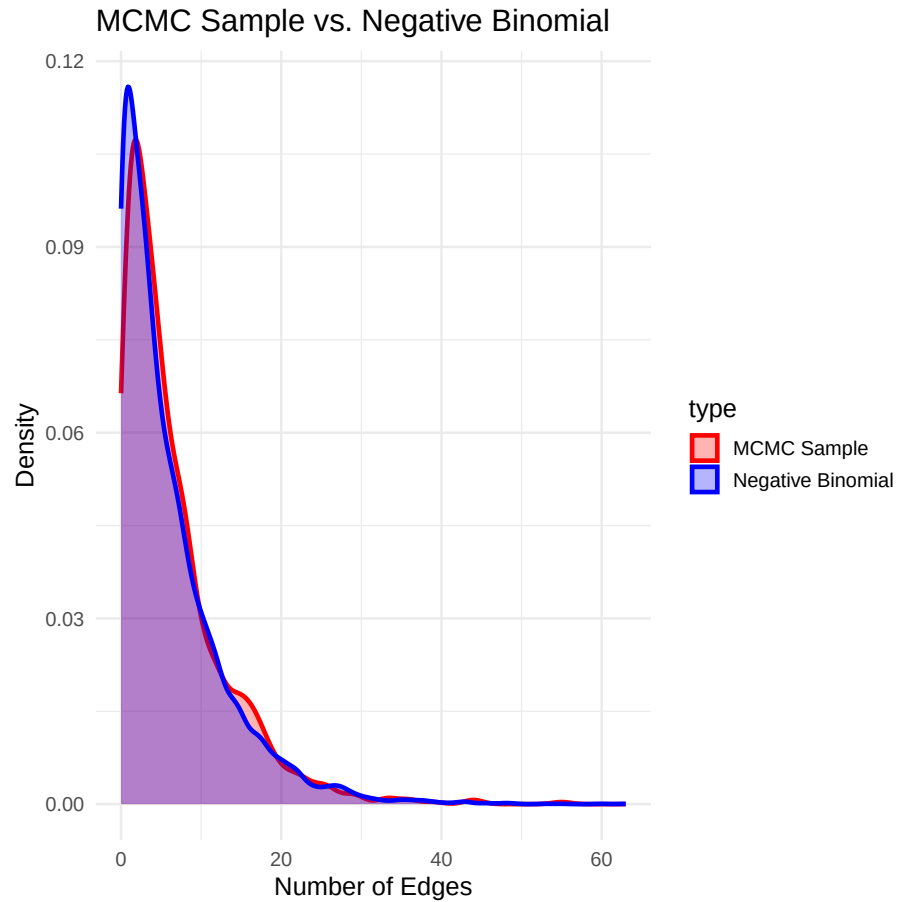
set.seed(123)
df_prior <- data.frame(
  value = rnbinom(10000, size = 1.017340, mu = 6.192894),
  type  = "Negative Binomial"
)

df_all <- bind_rows(df_sampled, df_prior)

ggplot(df_all, aes(x = value, color = type, fill = type)) +
  geom_density(alpha = 0.3, linewidth = 1.2) +
  labs(
    title = "MCMC Sample vs. Negative Binomial",
    x = "Number of Edges",
    y = "Density"
  ) +
  theme_minimal(base_size = 14) +

```

```
scale_color_manual(values = c("MCMC Sample" = "red", "Negative Binomial" = "blue")) +
scale_fill_manual(values = c("MCMC Sample" = "red", "Negative Binomial" = "blue"))
```



2 Example 2: Node Degree Distribution

```
# Population parameters
population <- 100
covPattern <- rep(0L, population)

# Network model parameters
Network_stats <- list(c("Degree"))
Prob_Distr <- list(c("Multinomial_Poisson"))
Prob_Distr_Params <- vector("list", 2)
Prob_Distr_Params[[1]] <- c(population)
```

```

Prob_Distr_Params[[2]] <- dnbinom(0:(population-1), size=1.017340, mu=6.192894)

samplesize <- as.integer(1000)
burnin <- as.integer(200000)
interval <- as.integer(1000)

CCMnet_Result <- CCMnetpy::CCMnet_constr(
  Network_stats=Network_stats,
  Prob_Distr=Prob_Distr,
  Prob_Distr_Params=Prob_Distr_Params,
  samplesize = samplesize,
  burnin=burnin,
  interval=interval,
  statonly=TRUE,
  G=NULL,
  P=NULL,
  population=as.integer(population),
  covPattern = as.integer(covPattern),
  bayesian_inference = FALSE,
  Ia = NULL,
  Il = NULL,
  R = NULL,
  epi_params = NULL,
  print_calculations = FALSE,
  use_G = FALSE,
  outfile = "none",
  partial_network = as.integer(0),
  obs_nodes = NULL,
  MH_proposal_type = "TNT"
)

CCM_stats <- CCMnet_Result[[2]]

degrees_to_check <- 0:9
theoretical_pmf <- Prob_Distr_Params[[2]]
n_sim <- 1000

# Empirical distribution
df_empirical <- CCM_stats %>%
  as.data.frame() %>%
  pivot_longer(cols = everything(),
               names_to = "degree",
               values_to = "count") %>%
  filter(degree %in% degrees_to_check) %>%
  group_by(degree) %>%

```

```

mutate(source = "MCMC")

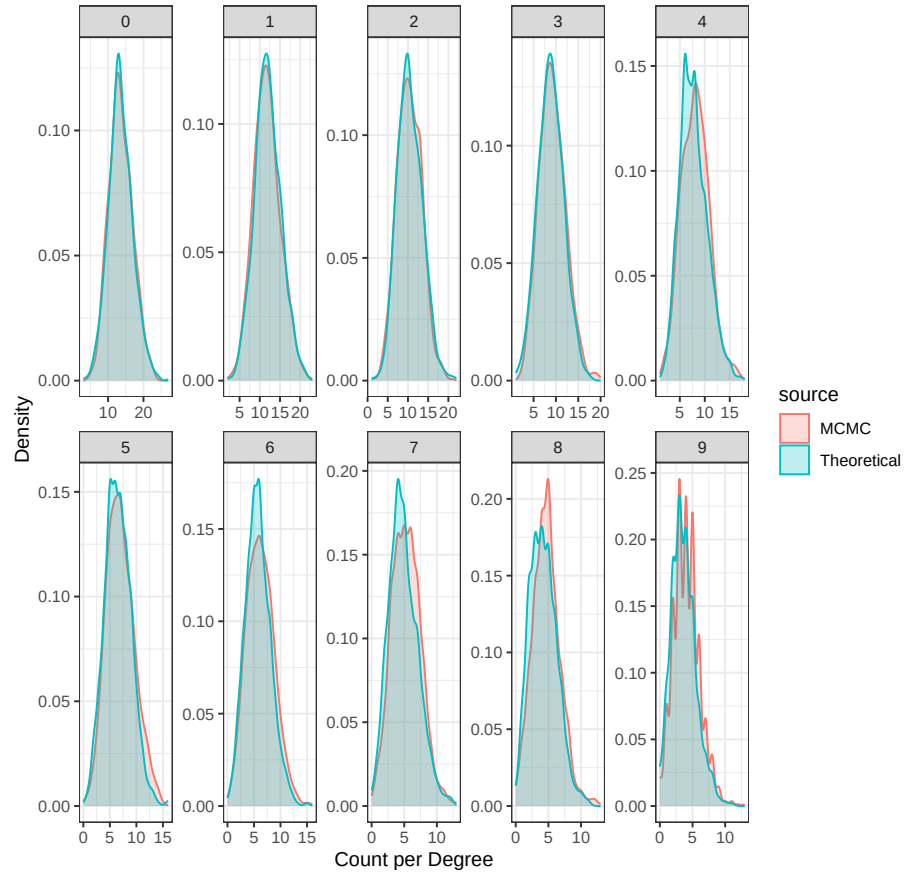
# Theoretical distribution
set.seed(123)
multi_pois <- t(rmultinom(n_sim, population, prob = theoretical_pmf))
df_theory <- multi_pois %>%
  as.data.frame()
colnames(df_theory) <- 0:(population-1)
df_theory <- df_theory %>%
  pivot_longer(cols = everything(),
               names_to = "degree",
               values_to = "count") %>%
  filter(degree %in% degrees_to_check) %>%
  group_by(degree) %>%
  mutate(source = "Theoretical")

# Combine and plot
df_plot <- bind_rows(df_empirical, df_theory)

ggplot(df_plot, aes(x = count, color = source, fill = source)) +
  geom_density(alpha = 0.25) +
  facet_wrap(~degree, scales = "free", ncol = 5) +
  theme_bw() +
  labs(
    title = "Degree Distribution: Empirical MCMC vs Theoretical Multinomial-Poisson (Degrees
    x = "Count per Degree",
    y = "Density"
  )

```

Degree Distribution: Empirical MCMC vs Theoretical Multinomial-Poisson (Degree



3 Example 3: Mixing Matrix Distribution

```
population <- 100
covPattern <- c(rep(0L, population/2), rep(1L, population/2))

Network_stats <- list(c("Mixing"))
Prob_Distr <- list(c("Multinomial_Poisson"))
Prob_Distr_Params <- vector("list", 2)
Prob_Distr_Params[[1]] <- c(20)
Prob_Distr_Params[[2]] <- c(.3, .6, .1)

samplesize <- as.integer(1000)
burnin <- as.integer(200000)
interval <- as.integer(1000)
```



```

CCMnet_Result <- CCMnetpy::CCMnet_constr(
  Network_stats=Network_stats,
  Prob_Distr=Prob_Distr,
  Prob_Distr_Params=Prob_Distr_Params,
  samplesize = samplesize,
  burnin=burnin,
  interval=interval,
  statsonly=TRUE,
  G=NULL,
  P=NULL,
  population=as.integer(population),
  covPattern = as.integer(covPattern),
  bayesian_inference = FALSE,
  Ia = NULL,
  Il = NULL,
  R = NULL,
  epi_params = NULL,
  print_calculations = FALSE,
  use_G = FALSE,
  outfile = "none",
  partial_network = as.integer(0),
  obs_nodes = NULL,
  MH_proposal_type = "TNT"
)

stats <- CCMnet_Result[[2]]
colnames(stats) <- c("M11", "M12", "M22")

# Simulate theoretical counts
multi_pois <- c()
for (i in 1:1000) {
  multi_pois <- bind_cols(multi_pois, rmultinom(1, rpois(1,20), prob = c(.3, .6, .1)))
}

## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:

```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## New names:
## * ' -> '...1'

multi_pois <- t(multi_pois)
```

```

colnames(multi_pois) <- c("M11", "M12", "M22")

# Long format
df_stats <- stats %>%
  as.data.frame() %>%
  mutate(source = "MCMC") %>%
  pivot_longer(cols = c(M11, M12, M22), names_to = "metric", values_to = "count")

df_pois <- multi_pois %>%
  as.data.frame() %>%
  mutate(source = "Poisson") %>%
  pivot_longer(cols = c(M11, M12, M22), names_to = "metric", values_to = "count")

df_all <- bind_rows(df_stats, df_pois)

ggplot(df_all, aes(x = count, color = source, fill = source)) +
  geom_density(alpha = 0.3) +
  facet_wrap(~metric, scales = "free") +
  theme_bw() +
  labs(
    title = "Distribution of Edge Counts: MCMC vs Poisson Multinomial",
    x = "Count",
    y = "Density"
  )

```


Distribution of Edge Counts: MCMC vs Poisson Multinomial

